When More Knowledge Is Not Better: Prompt-Level Knowledge Augmentation for NL2SQL in General-Purpose LLM Workflows

Jingjing Tian[0009−0008−3852−974X]

University of Liverpool, Liverpool L69 7ZX, United Kingdom
sgjtian5@liverpool.ac.uk

Abstract. Large language models (LLMs) show promise in text-to-SQL (NL2SQL) tasks, yet their performance remains sensitive to schema grounding and contextual knowledge design. This study empirically examines whether schema knowledge augmentation improves general-purpose NL2SQL workflow performance. We construct an alias mapping dictionary via a three-stage pipeline and integrate it into a general-purpose LLM workflow for prompt-level knowledge augmentation. The experiment across a single-table dataset and the robustness experiment reveal that unstructured prompt-level knowledge injection led to degradation in the main experiment and no significant improvement in the robustness experiment. Our results show that adding schema and alias knowledge as unstructured prompt text does not yield monotonic improvements in general-purpose NL2SQL workflows; effective augmentation requires task-aware mechanisms for representing and consuming knowledge. These findings challenge the assumption that more knowledge is uniformly beneficial, and call for NL2SQL-aware knowledge consumption mechanisms that preserve structural semantics.

Keywords: Text-to-SQL · Knowledge Augmentation · Schema Grounding · Large Language Models · LLM Workflow · Prompt Engineering.

1 Introduction

As LLM applications are increasingly adopted in enterprises, NL2SQL systems are being deployed in business intelligence to reduce the cost of ad-hoc SQL writing. However, schema ambiguity has been recognised as a challenge in NL2SQL, where natural language expressions may correspond to multiple candidate schema elements, such as columns or tables [1]. Knowledge augmentation is widely used and considered to have a positive effect on NL2SQL systems. Classic methods include direct injection into the LLM prompt, online retrieval from a knowledge base, and collaboration with another knowledge-searching agent [6]. [8] used an additional thesaurus dictionary for disambiguation. [3] convert documents from vertical business scenes to structural knowledge. Large-scale benchmarks like BIRD also include external knowledge as an indispensable component of NL2SQL evaluation [5]. However, prior work largely focuses on dedicated NL2SQL systems, whereas business intelligence practice often involves

small-schema tables deployed through low-code LLM workflows. Our contributions are threefold: (1) a controlled comparison of schema-only, schema-card, and alias-augmented prompting in a general-purpose workflow; (2) empirical evidence that richer knowledge does not monotonically improve EM or EX in compact single-table settings; and (3) a mechanism-level explanation that unstructured prompt-level consumption increases ambiguity rather than reducing it.

2 Related Work

Existing studies on LLM-based NL2SQL often assume that additional knowledge, such as schema description, context documents, and thesaurus, improves schema understanding and query generation. For example, [4] concatenate static text from a financial industry corpus into the prompt, [3] use RAG to retrieve text chunks from business documents into the prompt, [7] utilize the long-context capability of Gemini-1.5, put the full database schema into the prompt, [10] combined RAG and prompt engineering with a fine-tuned model. However, these studies mainly investigate how to add knowledge, while less attention has been paid to whether additional knowledge is always beneficial under general-purpose LLM workflow settings.

Beyond SQL grammar, a central challenge in NL2SQL is grounding natural-language expressions to the correct schema elements. Schema ambiguity, alias mismatch, dirty values, and noisy context can affect column selection. Prior work has addressed this issue through retrieval and ranking of schema elements [2], parser-independent schema disambiguation [1], thesaurus-based ambiguity resolution with gating mechanisms [8], and analyses of noise effects in NL2SQL benchmarks [11]. In specialised NL2SQL systems, such knowledge is often consumed through structured mechanisms, including explicit schema linking in RAT-SQL [15] and RESDSQL [13], or multi-step decomposition and self-correction in DIN-SQL [14]. These mechanisms bind natural-language phrases to schema elements before or during generation, mitigating the ambiguity in ways that plain-text prompt injection does not enforce. Building on this line of work, our study examines a practical question: when alias and schema knowledge are injected into a general-purpose LLM workflow as textual context, does this knowledge improve or degrade NL2SQL generation?

3 Methodology

3.1 Task and Experimental Design

We formulated the task as standard NL2SQL based on our research question: given a natural language question and a relational database schema, the system generates an executable SQL query. We fix all variables except the knowledge condition to isolate its effect. Three configurations are evaluated as presented in Table 1: a minimal baseline (v0), a schema-augmented variant (v1), and a

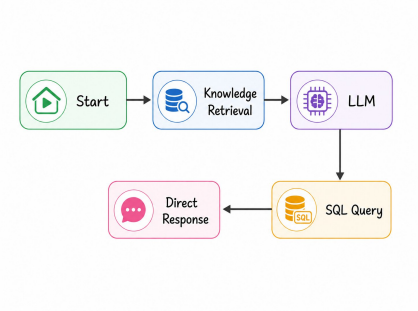


Fig. 1: Dify workflow illustration.

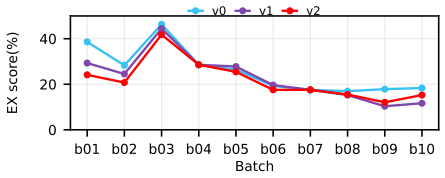


Fig. 2: Per-batch EX scores of the main experiment across ten evaluation batches.

schema-plus-alias-augmented variant (v2). We hypothesise that v2 should outperform v1, and v1 should outperform v0, if knowledge augmentation operates as expected. The settings and configurations are identical across three variants. The generation is conducted in a single-shot setting. Only the knowledge condition differs. Evaluation is conducted on a public dataset[1] implemented in MySQL. The input natural language questions are synthetically generated using templates and GPT-5.2-based paraphrasing to simulate realistic user queries. We use Qwen2.5-72B-Instruct to generate candidate gold SQL, which then passes a multi-stage validation pipeline decoupled from the generation step (detailed in Appendix A). Qwen3-Next-80B-A3B-Instruct serves as the production LLM in the Dify workflow. We set the temperature to 0.7 and otherwise use Dify defaults. For knowledge retrieval, we used BGE-M3 embeddings with a bge-reranker-m3 reranker. The retrieval node returns the top-3 ranked chunk. The details about the knowledge base construction and experiment setup can be found in A.1 and Table 6, respectively. EM, EX are computed as they are two mainstream performance evaluation metrics for NL2SQL tasks [6], with McNemar tests further restricted to valid paired outputs.

3.2 System and Knowledge Retrieval

The experiment system (Figure 1) was built on Dify[2], a general-purpose LLM workflow engine, and run locally in Docker. We deliberately use Dify, an off-the-shelf platform, to mirror real-world deployment constraints. The input to the LLM node comprises the user question and a system prompt that varies depending on the knowledge condition. We incorporate an alias mapping mechanism to form the three variants of the knowledge base (Table 1) for the workflow, where natural language expressions are matched to dataset schema elements through a predefined dictionary of synonyms. The knowledge retrieval node extracts information from the knowledge base and passes the result to the LLM node (Figure

[1] Movie dataset: `https://www.kaggle.com/datasets/danielgrijalvas/movies`
[2] Dify: an open-source LLM application development platform. `https://github.com/langgenius/dify`

Table 1: Knowledge conditions by variant.

Variant	Prompt Content
v0	Raw schema: movies(id, name, year, ...)
v1	v0 + Schema cards (description + sample values)
v2	v1 + Alias mapping dictionary (NL→column, confidence)

Table 2: Overall EM/EX performance and McNemar significance tests across system variants. $^*p < 0.05$, $^{**}p < 0.01$.

Variant	**EM (%)**	**EX (%)**	**Pair**	**EM** p	**EX** p
v0	10.5	25.6	v0:v1	0.677	0.021*
v1	9.9	22.7	v0:v2	0.006**	0.006**
v2	7.2	21.7	v1:v2	0.009**	0.560

1). In v0, the prompt contains the user question, raw table schema, database constraints, and SQL-only instructions. In v1, schema cards containing column descriptions and representative values are appended to the same template. In v2, retrieved alias chunks are further appended as plain-text context, including alias expressions, target columns, and confidence scores.

4 Results

Table 2 shows monotonic degradation: v0 achieves EM 0.105 / EX 0.256, v1 drops to 0.099 / 0.227, and v2 falls further to 0.072 / 0.217. McNemar tests confirm the v0 to v2 decline is significant (p=0.006 for both metrics), while v0 to v1 reaches significance only for EX (p=0.021) and v1 to v2 only for EM (p=0.009). The downward trend holds in 8/10 batches (Appendix Figure 3).

5 Analysis and Discussion

Table 3: Error taxonomy based on v0→v2 transition patterns.

Type	**Definition (v0 → v2)**	**Mechanism**
A	correct → wrong	Inference Over-Complexity
B	wrong → correct	Domain-Specific Disambiguation
C	all wrong	Common Bottleneck / Structural Deficit

To understand why knowledge augmentation did not improve NL2SQL performance, we inspected model outputs and group errors according to the transition patterns between v0 and v2 as presented in Table 3. In Type A cases, the LLM node receives additional retrieved knowledge and schema alias mapping with the v2 variant, attempting to match a more complex predicate logic, which resulted in redundant SQL being generated. In this case, knowledge expands the

inference space. With more information about the table schema and fields retrieved and input into the LLM node, the search space is also expanded. We refer to this failure mode as inference over-complexity: additional schema and alias context increase the number of plausible grounding paths, causing the model to over-specify otherwise simple queries.

v0 (correct): `SELECT director, MAX(score)-MIN(score) ... GROUP BY director LIMIT 1;`
v2 (wrong): `SELECT director, MAX(rating), MIN(rating), MAX(rating)-MIN(rating) ... HAVING COUNT(DISTINCT rating)>1 ...`

As the example SQL quoted above, v2, with the full knowledge augmentation, selected the wrong field rating for the input question "Which productions by the same director show the largest rating differences?", and added more filters and constraints in the output SQL. [11] also reveal that noisy or excessive knowledge can complicate model reasoning. Beyond the additional choices introduced by retrieved knowledge, how schema knowledge is consumed matters. As a general-purpose LLM workflow orchestration platform, Dify does not have an NL2SQL-specific knowledge process mechanism. Log analysis reveals that schema descriptions, alias mappings, and confidence scores are all injected into the prompt of the LLM node as plain text rather than machine-readable schema constraints that restrict column selection. The knowledge retrieval node ranks chunks by relevance to the user query and returns them with metadata scores, yet these confidence scores are embedded as literal text rather than consumed as structured priors over candidate columns. These factors caused the LLM to select the wrong column "rating", instead of the right column "score". This failure pattern is not limited to cases where v0 is correct. In an all-wrong case asking for the "biggest gap between average rating and average popularity", v0 and v1 preserved the intended `score`/`votes` mapping but produced non-canonical formulations with extra projections and an absolute difference. By contrast, v2 shifted the grounding to `rating`/`score`, despite a lower-ranked retrieved chunk pointing to `votes`. This suggests that prompt-level knowledge injection can change the type of failure, not merely the correctness outcome. Integrating task-specific knowledge (e.g., schema constraints and domain rules) into LLM is an open challenge as current methods still rely on limited prompts or manual prompt engineering [12].

Additional knowledge is not uniformly harmful. In Type B cases, v2 corrects some column-selection errors made by v0. By learning additional schema-related and domain-specific knowledge via prompt injection, the LLM node successfully revised its incorrect column-selection decision in v2. For example, the query "filmmakers with the most productions" leads v0 to select `'company'` (no direct match), whereas v2 correctly grounds `filmmakers` to `'director'` column via the alias dictionary. Similarly, "commercially successful movies" causes v0 to inject a redundant `budget > 0` constraint, while v2 precisely maps to `gross > 0`. However, due to the limited complexity of our selected dataset, which has only 16 columns, the alias mapping system and the knowledge base it enhanced did not demonstrate their full augmentation potential in the Dify workflow. For

Table 4: Retrieval-level diagnostic example: "Which productions by the same director show the largest rating differences?"

Retrieval Score	Alias Content	Target Column	Alias Confidence
1.0	productions fall below their genres usual rating despite high visibility	movies.rating	0.8436
0.535	productions rated higher than	movies.rating	0.8194
0.0	score differ between older	movies.score	0.9071

many business scenarios involving small datasets like this one, other methods such as fine-tuning, NL2SQL-specific models, or manual validation may be more suitable. An explicit example is the NL2SQL scenario provided in [9]'s work, which contains 15 relations with an average of 2000 records and 35–65 fields in each relation, focused on user questions from real business scenarios in the power industry.

To examine whether the observed degradation was dataset-specific, an additional robustness experiment was conducted on a wider Short-video dataset[3]. Unlike the main Movie dataset experiment, the second dataset showed only small numerical differences: EM declined from 0.494 in v0 to 0.471 in v2, while EX remained nearly unchanged from 0.647 to 0.641 (Figure 4 and Table 7). Paired McNemar tests showed no statistically significant differences among the three variants. These results suggest that prompt-level schema and alias injection do not consistently improve NL2SQL performance. The higher absolute scores may reflect differences in query distribution, schema design, and validation coverage rather than an inherent benefit of the dataset.

6 Conclusion

This paper examined whether prompt-level knowledge augmentation improves NL2SQL performance in a general-purpose LLM workflow. Across the main experiment and a robustness experiment, adding schema cards and alias mappings did not yield monotonic improvements over the raw-schema baseline. Our analysis suggests that the limitation lies not in knowledge availability alone, but in how retrieved knowledge is consumed: when schema and alias signals are injected as unstructured prompt text, they may expand the model's grounding space and induce over-specification or semantic drift. This suggests that general-purpose LLM workflow orchestration platforms require NL2SQL-aware mechanisms before retrieved schema knowledge can be reliably used for SQL generation. The knowledge base focused on schema alignment and ambiguity elimination may

[3] YouTube/TikTok Trends Dataset: `https://www.kaggle.com/datasets/tarekmasryo/youtube-shorts-and-tiktok-trends-2025/data?select=youtube_shorts_tiktok_trends_2025.csv`

prove more beneficial on larger datasets with greater schema complexity. Future work could focus on mechanisms that enable LLMs to consume schema knowledge and alias mappings as schema-aware structural information rather than plain textual context, and evaluate whether such mechanisms improve robustness on larger multi-table schemas.

Limitations

Our main experiment was conducted on a single-table movie dataset with a relatively compact schema, which aims to investigate how knowledge augmentation performs on NL2SQL tasks in scenarios where small datasets are involved. While such settings are common in internal BI dashboards and narrow-domain analytics, future work should validate whether the observed degradation persists on multi-table benchmarks such as BIRD [5], with richer schema complexity and cross-table reasoning demands.

The absolute EX/EM scores were low, which might be caused by using exact single-shot generation against validated gold SQL without execution-guided repair, candidate reranking, or manual post-correction. The focus of this study is not to establish a competitive NL2SQL system or an effective knowledge base building pipeline, but to compare NL2SQL performance changes under controlled knowledge conditions.

We have not conducted experiments based on other structured-consumption NL2SQL systems due to limited resources. We hypothesised that structured consumption like the JSON schema with constrained column selection would mitigate the observed performance degradation in our experiments. Empirical validation is left to future work.

References

1. Zhang, M., Ma, K., Xu, L., Zhang, K., Peng, Y., Jin, R.: CLEAR: A Parser-Independent Disambiguation Framework for NL2SQL. In: 2025 IEEE 41st International Conference on Data Engineering (ICDE), pp. 1–14. IEEE, (2025). https://doi.org/10.1109/ICDE65448.2025.00247
2. Weng, L., Tang, Y., Feng, Y., Chang, Z., Chen, R., Feng, H., Hou, C., Huang, D., Li, Y., Rao, H., Wang, H., Wei, C., Yang, X., Zhang, Y., Zheng, Y., Huang, X., Zhu, M., Ma, Y., Cui, B., Chen, P., Chen, W.: DataLab: A Unified Platform for LLM-Powered Business Intelligence. In: 2025 IEEE 41st International Conference on Data Engineering (ICDE), pp. 4346–4359. IEEE, (2025). https://doi.org/10.1109/ICDE65448.2025.00326
3. Zhao, A., Li, R., Fu, B.: Enhancing ChatBI System Performance in Vertical Business Domains through an Automated Sample Generation Pipeline. In: 2024 7th International Conference on Universal Village (UV), pp. 1–10. IEEE, (2024). https://doi.org/10.1109/UV63228.2024.11189210
4. Du, X., Hu, S., Zhou, F., Wang, C., Nguyen, B. M.: FI-NL2PY2SQL: Financial Industry NL2SQL Innovation Model Based on Python and Large Language Model. Future Internet **17**(1), 12 (2025). https://doi.org/10.3390/fi17010012

5. Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K. C. C., Huang, F., Cheng, R., Li, Y.: Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. Advances in Neural Information Processing Systems **36**, (2023).
6. Liu, X., Shen, S., Li, B., Ma, P., Jiang, R., Zhang, Y., Fan, J., Li, G., Tang, N., Luo, Y.: A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and Where Are We Going? IEEE Transactions on Knowledge and Data Engineering **37**(10), 5735–5754 (2025). https://doi.org/10.1109/TKDE.2025.3592032
7. Chung, Y., Kakkar, G. T., Gan, Y., Milne, B., Özcan, F.: Is Long Context All You Need? Leveraging LLM's Extended Context for NL2SQL. Proceedings of the VLDB Endowment **18**(8), 2735–2747 (2025). https://doi.org/10.14778/3742728.3742761
8. Cai, Q., Yu, X., Chen, X., Guo, W.: NL2SQL Method Based on Thesaurus and Knowledge-Augmented. In: 2025 IEEE 6th International Conference on Computer, Big Data, Artificial Intelligence (ICCBD+AI), pp. 1–5. IEEE, (2025). https://doi.org/10.1109/ICCBDAI66607.2025.11388593
9. Huang, B., Zhang, J., Sun, Y., Luo, W., Guo, Y., Hu, S., Wang, K., Zhao, Y.: NL2SQL Method in the Power Domain Based on the Multi-Layer Collaborative Optimization Strategy. In: 2025 International Conference on Electrical, Control and Artificial Intelligence (ICOECAI), pp. 274–280. IEEE, (2025). https://doi.org/10.1109/ICOECAI67333.2025.11335997
10. Zhang, J., Li, Y., Liu, Y., Xie, L.: Research and Practice of NL2SQL Technology Based on LLM for Big Data of Enterprise Finance. In: 2024 4th International Conference on Advanced Enterprise Information System (AEIS), pp. 46–51. IEEE, (2024). https://doi.org/10.1109/AEIS65978.2024.00015
11. Wretblad, N., Riseby, F., Biswas, R., Ahmadi, A., Holmström, O.: Understanding the Effects of Noise in Text-to-SQL: An Examination of the BIRD-Bench Benchmark. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pp. 356–369. ACL, (2024). https://doi.org/10.18653/v1/2024.acl-short.34
12. Zhao, P., Shao, M., Cui, J., Zhou, H.: A Survey of NL2SQL Research: From Foundations to Frontiers. In: 2025 2nd International Conference on Intelligent Perception and Pattern Recognition (IPPR), pp. 396–402. IEEE, (2025). https://doi.org/10.1109/IPPR66507.2025.11198387
13. Li, H., Zhang, J., Li, C., Chen, H.: RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. Proceedings of the AAAI Conference on Artificial Intelligence **37**(11), 13067–13075 (2023). https://doi.org/10.1609/aaai.v37i11.26535
14. Pourreza, M., Rafiei, D.: DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In: Advances in Neural Information Processing Systems 36, pp. 36339–36348. Curran Associates, (2023). https://doi.org/10.52202/075280-1577
15. Wang, B., Shin, R., Liu, X., Polozov, O., Richardson, M.: RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 7567–7578. ACL, (2020). https://doi.org/10.18653/v1/2020.acl-main.677

7 Appendix

7.1 Alias Dictionary Construction

Following pipeline was adopted to construct the alias-to-schema mapping dictionary for NL2SQL schema grounding, which maps colloquial expressions, abbreviations, and synonymous references in natural language onto target columns in the database schema. The pipeline consists of three steps.

Step 1: High-Recall Candidate Retrieval The pipeline first normalizes alias phrases via lowercasing, tokenization, underscore/camelCase segmentation, and stopword filtering. BM25-based lexical retrieval is applied over enriched schema cards that concatenate column names, descriptions, and sample values. In this implementation, sample values are incorporated into the schema-card text, allowing lexical retrieval to capture additional cues such as country codes, percentage signs, and currency symbols.

Semantic Re-Ranking Both the alias and the corresponding column card, composed of the column name, description, and representative values, are encoded into dense vectors to compute semantic similarity. This semantic score is fused with lexical scores and hint signals to produce an overall confidence estimate. A threshold-and-margin based ambiguity detection mechanism is subsequently applied to distinguish high-confidence mappings from mappings requiring further review.

Lightweight Weak-Supervision Calibration Weak labels were derived from Question–SQL pairs and used to calibrate the fusion weights and confidence threshold, while Top-N/Top-K settings were treated as fixed experimental parameters. The final output is a reproducible alias dictionary that records each alias, its target table and column, a confidence score, and supporting evidence.

Table 5: Examples of alias mappings used in the alias dictionary.

Alias Phrase	Target Column	Conf.
audience score	movies.score	0.9218
audience votes	movies.votes	0.9388
interaction rates but remain trending	youtube_curated.trend_duration_days	0.8569
comment activity	youtube_curated.comment_ratio	0.8690

7.2 Gold SQL Validation Pipeline

Candidate gold SQL produced by the generation model is not used directly as ground truth. Each candidate passes a multi-stage validation pipeline:

Table 6: Experimental configuration for the main and robustness experiments.

Configuration	Main Experiment	Robustness Experiment
Target table	`Movies`	`Short video`
Schema size	1 table, 16 columns	1 table, 32 columns
Initial input questions	600	180
Gold Q–SQL retained after validation	568	170
Validation set Q–SQL pairs	60	90
Total unique alias phrases	103	159
Number of batches	10	6
Input per batch	b01–b10, 60 each; 600 in total	b01–b06, 30 each; 180 in total
Total successful SQL outputs across all runs	1,789	540

Table 7: Overall EM/EX performance and McNemar significance tests across system variants on the short-video dataset. $^{*}p < 0.05$, $^{**}p < 0.01$.

Variant	EM (%)	EX (%)	Pair	EM p	EX p
v0	49.4	64.7	v0:v1	0.838	0.700
v1	48.2	66.5	v0:v2	0.502	1.000
v2	47.1	64.1	v1:v2	0.683	0.289

Table 8: Error-type distribution across the Movies and Short-video datasets. Common denotes commonly aligned evaluation cases. A denotes cases where v0 is correct but v2 is wrong; B denotes cases where v0 is wrong but v2 is correct; C denotes cases where all variants are wrong; Uncl. denotes unclassified cases.

Dataset	Common	A	B	C	Uncl.
Movies	558	37	16	391	114
Short-video	170	11	10	45	104

Table 9: Average total token usage across system variants.

Experiment	Variant	Avg. Tokens	Runs with Usage
Main	v0	938.46	594
	v1	1439.64	597
	v2	1320.62	598
Robustness	v0	943.82	180
	v1	2589.18	180
	v2	2611.69	180

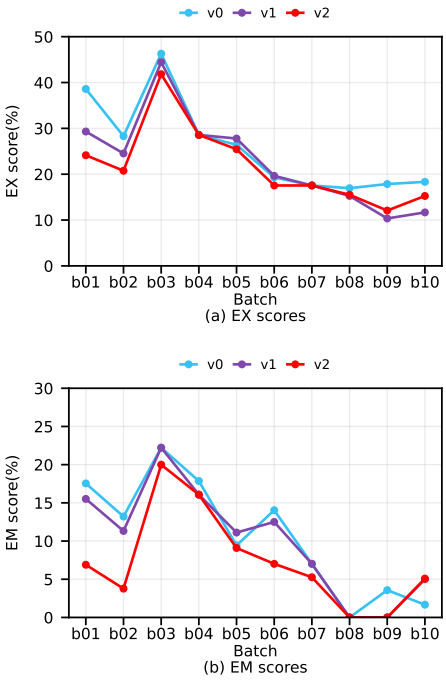


Fig. 3: Per-batch performance of Movie dataset across the ten evaluation batches under both EX and EM metrics.

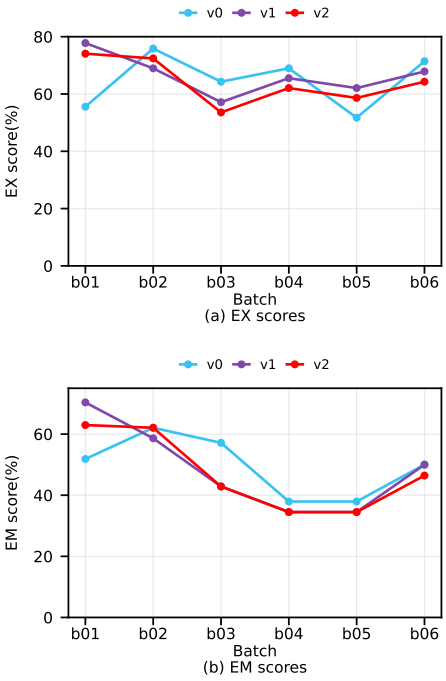


Fig. 4: Per-batch performance of the short-video dataset across the six evaluation batches under both EX and EM metrics.

(1) Hard-compliance and execution validation. Every candidate is first checked against deterministic, model-agnostic hard rules: it must be a single SELECT statement referencing only the allowed table, with no forbidden DDL/DML keywords or multiple statements. Candidates passing these static checks are then executed against the real MySQL database, and are retained only if execution succeeds and returns a well-formed result set.

(2) Rule-based structural screening. Executable candidates are further screened by deterministic rules that flag suspicious structures, such as full-table scans, missing aggregation, or a missing GROUP BY clause where the question implies grouping. These flags are produced by rules rather than by any model; flagged candidates are routed to a review stage rather than directly accepted.

(3) Answerability and validity review. Flagged candidates are reviewed for answerability and ground-truth validity. Cases judged unanswerable or requiring human judgment are escalated rather than silently accepted.

(4) Human-review pool. Cases that remain unresolved after the above stages are routed to a separate human-review pool rather than being automatically accepted as gold.

This validation relies primarily on model-agnostic checks (hard-compliance and execution) and deterministic structural rules, rather than on model judgment alone.

7.3 Example LLM node Input for v2 (Schema + Alias) in the main experiment

v0 (Raw Schema) — Base Template. The following system prompt is shared by all variants. User questions are appended as plain text after the schema.

```
You are an SQL expert for MySQL 8.0.

DATABASE CONSTRAINT (MUST FOLLOW):
- There is ONLY ONE table: `movies`.
- Any other table names DO NOT exist and MUST NOT be used.
- If a question cannot be answered using table `movies`, output exactly:
  SELECT NULL WHERE 1=0

SCHEMA:
Table: movies
Columns:
- id (BIGINT)
- name (VARCHAR)
...
- runtime (INTEGER)

STRICT RULES:
- Output exactly ONE executable SQL SELECT statement.
- ...

Question: {user_question}
```

v1 (Schema Cards) — Diff against v0. Identical to v0, except the SCHEMA section is replaced with column descriptions and representative values, e.g.:

```
- name (VARCHAR): Title of the movie. Examples: "The Godfather".
- rating (VARCHAR): MPAA content rating. Examples: "R", "PG-13".
...
```

v2 (Schema + Alias) — Diff against v1. Identical to v1, except the top-3 retrieved alias chunk is appended as plain text immediately after the schema cards:

```
Alias: {retrieved_natural_language_expression}
Target Column: movies.{column_name}
Confidence: {score}
```

Example actual LLM input (v2). The following shows the real system prompt assembled by Dify for the query *"Which productions by the same director show the largest rating differences?"* (1,365 tokens, Qwen3-Next-80B-A3B-Instruct):

```
You are an SQL expert for MySQL 8.0.
Given the user question: Which productions by the same director
show the largest rating differences? ,
and your knowledge of this database: [
  {'score': 1.0, 'content': 'Alias: productions fall below...
   \nTarget Column: movies.rating\nConfidence: 0.8436'},
  {'score': 0.535, 'content': 'Alias: productions rated higher than
   \nTarget Column: movies.rating\nConfidence: 0.8194'},
  {'score': 0.0, 'content': 'Alias: score differ between older
   \nTarget Column: movies.score\nConfidence: 0.9071'}
], output ONLY ONE executable SQL query.

[DATABASE CONSTRAINT and SCHEMA CARDS identical to v0/v1 above]
```